



3

Functions

Uwe R. Zimmer - The Australian National University

Functions

(Greatness from ...) Small beginnings

```
plus_1 :: (Num a) => a -> a
plus_1 x = x + 1
int_plus1 (int x) {
    return x + 1;
}
function plus_1 (x : Integer) return Integer is (x + 1);
def plus1 (x):
    return x + 1;
pure function plus_1 (x)
int, intent (in) :: x
int
    :: plus_1
plus_1 = x + 1;
end function;
function Plus_1 (x : Integer) : Integer;
begin
    Plus_1 := x + 1;
end;
```

Functions

esp How is the parameter x passed?

```
...
mov r0, #1
bl Plus_1
mov r4, r0
...
```

esp Does it work?

```
Plus_1:
add r0, #1
bx lr
```

esp Could it be done differently?

esp Where do you find the result after the function returns?

Functions

References for this chapter

[Patterson17]
David A. Patterson & John L. Hennessy
Computer Organization and Design – The Hardware/Software Interface
Chapter 2 "Instructions: Language of the Computer"
ARM edition, Morgan Kaufmann 2017

Functions

```
...
mov r0, #1
bl Plus_1
mov r4, r0
...
```

```
Plus_1:
add r0, #1
bx lr
```

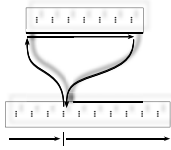
Functions

```
...
mov r0, #1
add r0, #1
mov r4, r0
...
```

esp This is called inlining

esp Can/should this always be done?

Functions

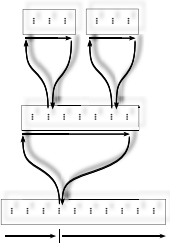


Functions

```
...
mov r0, #1
bl Plus_1
mov r4, r0
...
```

```
Plus_1:
add r0, #1
bx lr
```

Functions



Functions

(Greatness from ...) Small beginnings

```

plus_2 :: (Num a) => a -> a
plus_2 x = plus_1 $ plus_1 x

int_plus2 (int x) {
  return plus1 (plus1 (x));
}

function Plus_2 (x : Integer) return Integer is (Plus_1 (Plus_1 (x)));

def plus2 (x):
  return plus1 (plus1 (x));

pure function plus_2 (x)
int, intent (in) :: x
:: plus_2
plus_2 = plus_1 (plus_1 (x));
end function;

function Plus_2 (x : Integer) : Integer;
begin
  Plus_2 := Plus_1 (Plus_1 (x));
end;

```

Functions

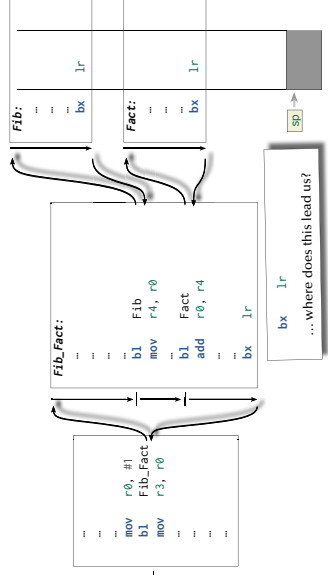
```

...
mov r0, #1
add r0, #2
mov r3, r0
...

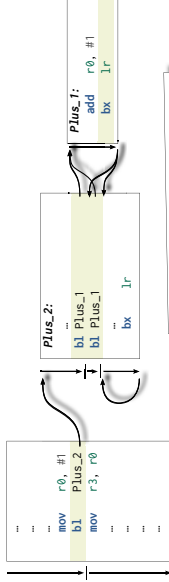
```

... we need an example, where a compiler will not just remove all our code!

Functions



Functions



bx lr
is used three times
see What is the value of lr in each case?

Functions

(Greatness from ...) Small beginnings

```

fib.fact :: (Num a) => a -> a
fib.fact x = (fib x) + (fact x)

unsigned int fibFact (unsigned int x) {
  return fib (x) + fact (x);
}

function Fib_Fact (x : Natural) return Natural is (fib (x) + fact (x));

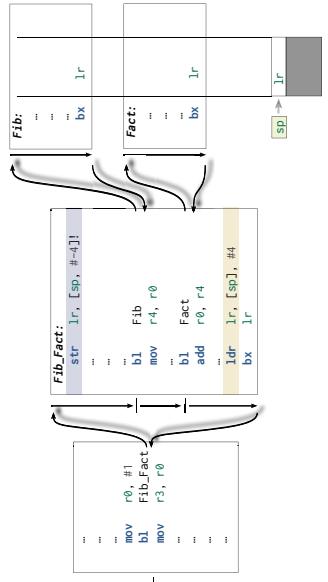
def fibFact (x):
  return fib (x) + fact (x);

pure function fib.fact (x)
int, intent (in) :: x
:: fib.fact
fib.fact = fib (x) + fact (x);
end function;

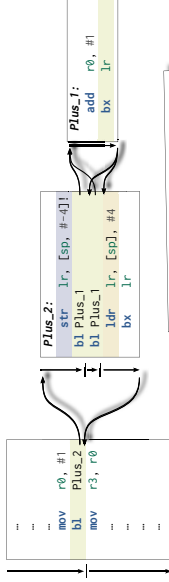
function Fib_Fact (x : cardinal) : cardinal;
begin
  Fib_Fact := Fib (x) + Fact (x);
end;

```

Functions

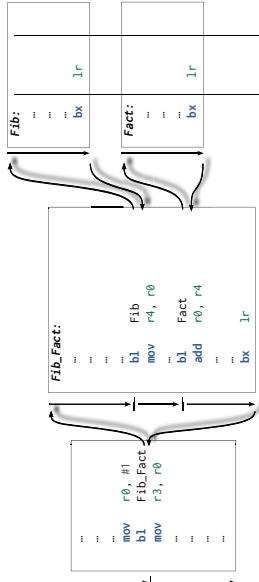


Functions

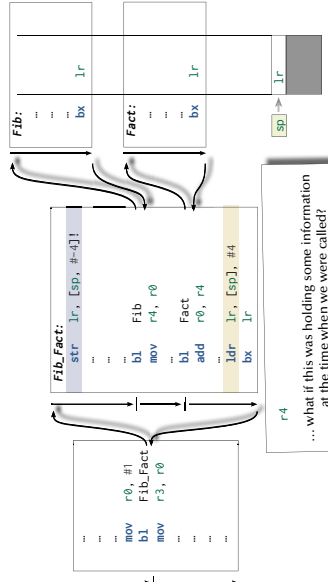


bx lr
is used three times
see What is the value of lr in each case?

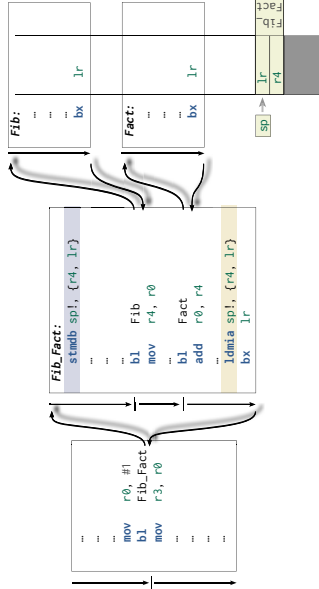
Functions



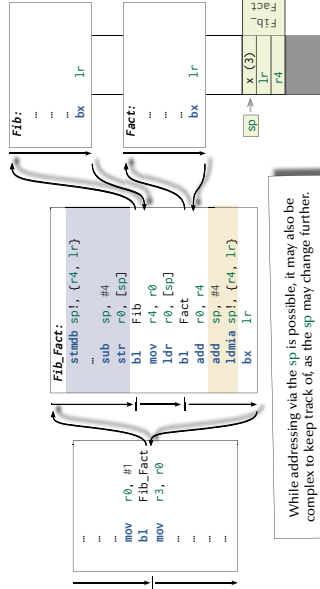
Functions



Functions

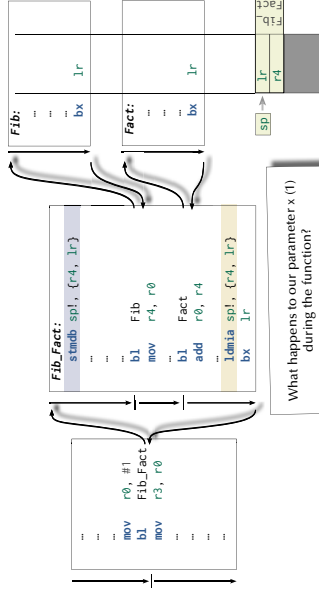


Functions



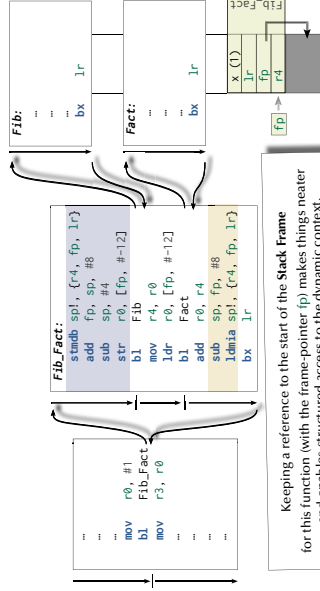
While addressing via the sp is possible, it may also be complex to keep track of, as the sp may change further.

Functions



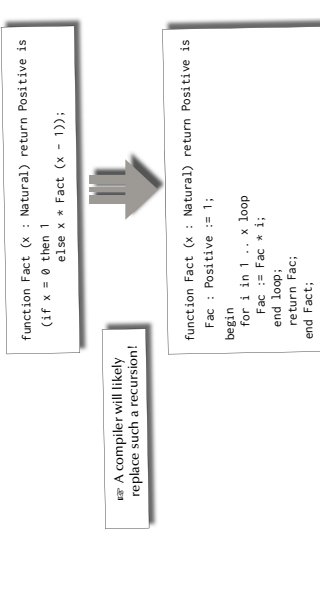
What happens to our parameter x (1) during the function?

Functions



Keeping a reference to the start of the Stack Frame for this function with the frame-pointer (fp) makes things neater and enables structured access to the dynamic context.

Functions

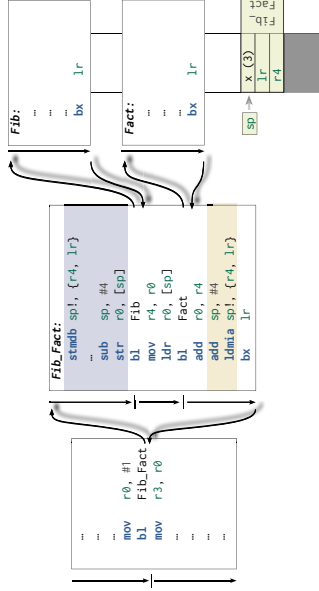


A compiler will likely replace such a recursion!

Function Fact (x : Natural) return Positive is
(if x = 0 then 1
else x * Fact (x - 1));

function Fact (x : Natural) return Positive is
Fac : Positive := 1;
begin
for i in 1 .. x loop
Fac := Fac * i;
end loop;
return Fac;
end Fact;

Functions



Functions

Recursive

```
unsigned int fib (unsigned int x) {
  switch (x) {
    case 0 : return 0;
    case 1 : return 1;
    default : return fib (x - 1) + fib (x - 2);
  }
}
```

function Fib (x : Natural) return Natural is

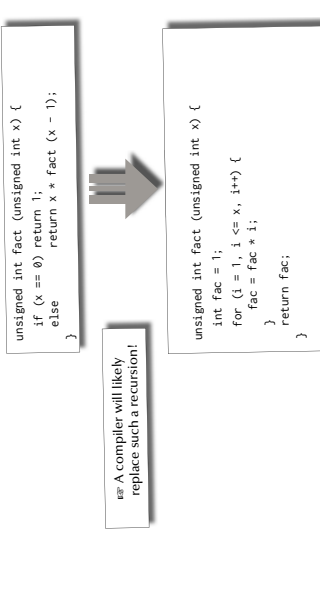
```
(case x is
  when 0 => 0;
  when 1 => 1;
  when others => Fib (x - 1) + Fib (x - 2));
```

unsigned int fact (unsigned int x) {
if (x == 0) return 1;
else
return x * fact (x - 1);
}

Function Fact (x : Natural) return Positive is
(if x = 0 then 1
else x * Fact (x - 1));

Will our stack handling work for recursive functions?

Functions



A compiler will likely replace such a recursion!

unsigned int fact (unsigned int x) {
if (x == 0) return 1;
else
return x * fact (x - 1);
}

unsigned int fact (unsigned int x) {
int fac = 1;
for (i = 1, i <= x, i++) {
fac = fac * i;
}
return fac;
}

Functions

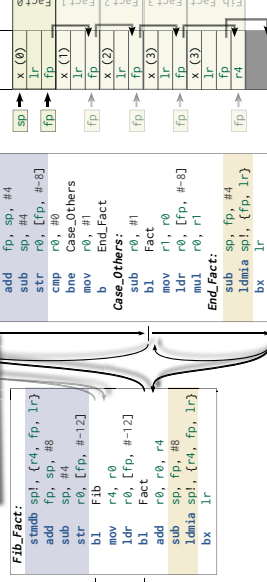
Ada

Limited control over "by value result".
"by value" parameters are constants.

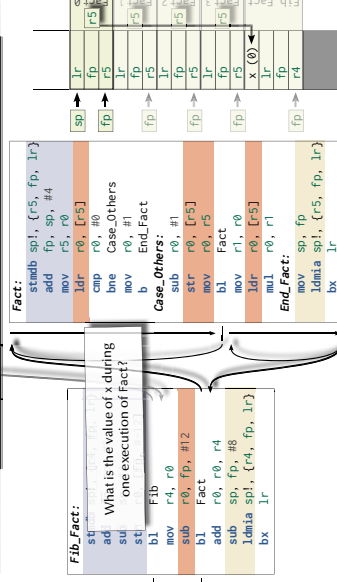
Information flow	Access	
	by copy	by reference
in	by value Parameter becomes a constant inside the function or is copied into a local variable.	by reference (immutable) No write access is allowed while the function runs (also from outside the function).
out	by result Calling function expects the parameter value to appear in a specific space at return.	by reference (mutable, no read) No read access from inside the function, write access on return.
in & out	by value result Parameter is copied to a local variable and copied back at return.	by reference (mutable) Function can read and write at any time. Outside code shall not write.

Functions

☞ How about using call-by reference here?



Functions



Functions

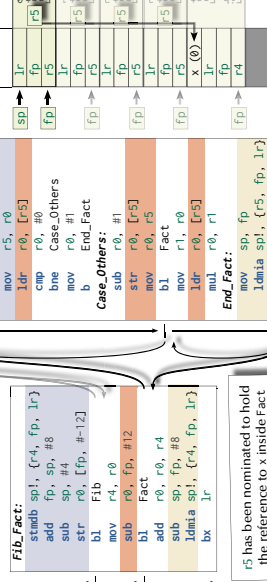
Assembly

"By reference" semantics by convention only.

Information flow	Access	
	by copy	by reference
in	by value Parameter becomes a constant inside the function or is copied into a local variable.	by reference (immutable) No write access is allowed while the function runs (also from outside the function).
out	by result Calling function expects the parameter value to appear in a specific space at return.	by reference (mutable, no read) No read access from inside the function, write access on return.
in & out	by value result Parameter is copied to a local variable and copied back at return.	by reference (mutable) Function can read and write at any time. Outside code shall not write.

Functions

☞ r5 has been nominated to hold the reference to x inside Fact



Functions

Parameter passing

Call by ...

Information flow	Access	
	by copy	by reference
in	by value Parameter becomes a constant inside the function or is copied into a local variable.	by reference (immutable) No write access is allowed while the function runs (also from outside the function).
out	by result Calling function expects the parameter value to appear in a specific space at return.	by reference (mutable) We should have used either of those modes no read access from inside the function, write access on return.
in & out	by value result Parameter is copied to a local variable and copied back at return.	by reference (mutable) Function can read and write at any time. Outside code shall not write.

Yet we used this mode

Functions

Parameter passing

Call by name

... is conceptually a call-by-value, where the value has not been calculated yet.
Technically a reference to a function is passed and the evaluation of this parameter (function) is left to the called function.

Features:

- Values are only evaluated if and when they are needed.
- Values can change during the life-time of a function (in case of side-effecting functions).
- Values can be stored once calculated (in case of side-effect-free functions).

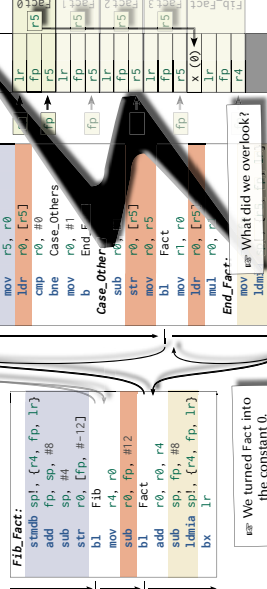
While this is possible to a degree in most programming languages ... (even if there is no specific passing mode, you can still pass a reference to a function)

... it is a core concept for functional, lazy evaluation languages, like e.g. Haskell,

and it does find its way back into mainstream languages like C++, .NET languages or Python as **anonymous functions** (sometimes referred to as λ -functions or λ -expressions).

Functions

☞ We turned Fact into the constant 0.



Functions

When to use what?

Information flow	Access	
	by copy	by reference
in	by value Parameter becomes a constant inside the function or is copied into a local variable.	by reference (immutable) No write access is allowed while the function runs (also from outside the function).
out	by result Calling function expects the parameter value to appear in a specific space at return.	by reference (mutable, no read) No read access from inside the function, write access on return.
in & out	by value result Parameter is copied to a local variable and copied back at return.	by reference (mutable) Function can read and write at any time. Outside code shall not write.

One-way and by-copy

Those are side-effect-free and hence the resulting scenarios are easy to analyse.
Copying large data structures might be time consuming or infeasible.
Values can be passed in registers.

		Access	
Information flow		by copy	by reference
		by value	by reference (immutable)
in		Parameter becomes a constant inside the function or is copied into a local variable.	No write access is allowed while the function runs (also from outside the function).
out		by result	by reference (mutable, no read)
		Calling function expects the parameter value to appear in a specific space at return.	No read access from inside the function, write access on return.
in & out		by value result	by reference (mutable)
		Parameter is copied to a local variable and copied back at return.	Function can read and write at any time. Outside code shall not write.

One-way-Out and by-reference

Side-effect-free, if new memory is allocated on return
– cannot be enforced on assembly level (requires compiler).
Values have to be passed in memory.

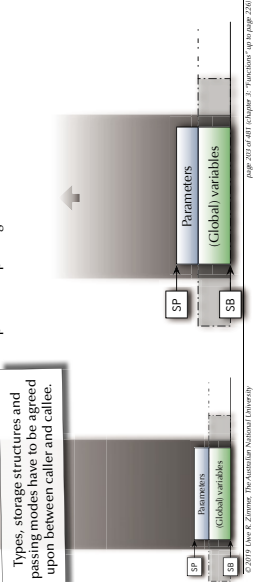
		Access	
Information flow		by copy	by reference
		by value	by reference (immutable)
in		Parameter becomes a constant inside the function or is copied into a local variable.	No write access is allowed while the function runs (also from outside the function).
out		by result	by reference (mutable, no read)
		Calling function expects the parameter value to appear in a specific space at return.	No read access from inside the function, write access on return.
in & out		by value result	by reference (mutable)
		Parameter is copied to a local variable and copied back at return.	Function can read and write at any time. Outside code shall not write.

Functions

Generic Stack-Frame

The current code prepared to call a function:

- Push parameters on the stack.
- Works for any data size (unless the stack overflows) and parameter passing mode.



Two-way and by-copy

Still side-effect-free within the function (but not on the outside).
Potentially more convenient as memory space can be reused.
Values can be passed in registers.

		Access	
Information flow		by copy	by reference
		by value	by reference (immutable)
in		Parameter becomes a constant inside the function or is copied into a local variable.	No write access is allowed while the function runs (also from outside the function).
out		by result	by reference (mutable, no read)
		Calling function expects the parameter value to appear in a specific space at return.	No read access from inside the function, write access on return.
in & out		by value result	by reference (mutable)
		Parameter is copied to a local variable and copied back at return.	Function can read and write at any time. Outside code shall not write.

One-way-In and by-reference

Side-effect-free – cannot be enforced on assembly level (requires compiler).
No data has to be replicated.
Values have to be passed in memory.

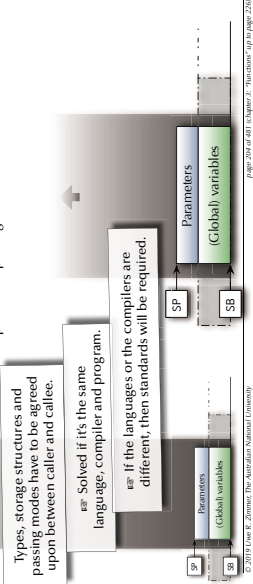
		Access	
Information flow		by copy	by reference
		by value	by reference (immutable)
in		Parameter becomes a constant inside the function or is copied into a local variable.	No write access is allowed while the function runs (also from outside the function).
out		by result	by reference (mutable, no read)
		Calling function expects the parameter value to appear in a specific space at return.	No read access from inside the function, write access on return.
in & out		by value result	by reference (mutable)
		Parameter is copied to a local variable and copied back at return.	Function can read and write at any time. Outside code shall not write.

Functions

Generic Stack-Frame

The current code prepared to call a function:

- Push parameters on the stack.
- Works for any data size (unless the stack overflows) and parameter passing mode.



Two-way and by-reference

Side-effecting and particular care is required as multiple entities could write on this.
No data has to be replicated.
Values have to be passed in memory.

		Access	
Information flow		by copy	by reference
		by value	by reference (immutable)
in		Parameter becomes a constant inside the function or is copied into a local variable.	No write access is allowed while the function runs (also from outside the function).
out		by result	by reference (mutable, no read)
		Calling function expects the parameter value to appear in a specific space at return.	No read access from inside the function, write access on return.
in & out		by value result	by reference (mutable)
		Parameter is copied to a local variable and copied back at return.	Function can read and write at any time. Outside code shall not write.

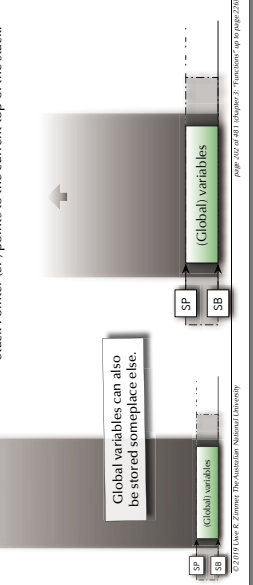
Functions

Generic Stack-Frame

Let there be some (global) data on the stack.

Stack-Base (SB) is a static address, always pointing to the ... well.

Stack-Pointer (SP) points to the current top of the stack.



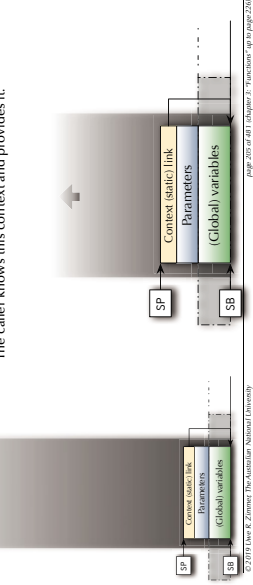
Functions

Generic Stack-Frame

Functions (in programming languages) have a context.

- E.g. the surrounding function or the hosting object.

The caller knows this context and provides it.



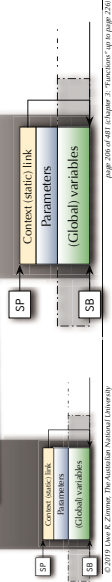
Functions

Generic Stack-Frame

Functions (in programming languages) have a context.
 ↳ E.g. the *surrounding function* or the *hosting object*.
 The caller knows this context and provides it.

↳ Some languages will not have a context by default, like C or Assembly.

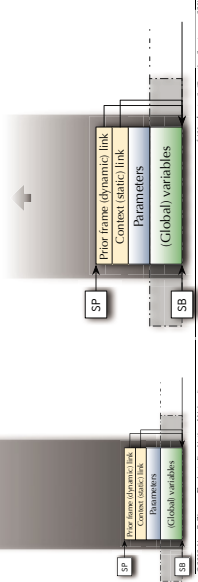
(gnu C expands the C standard and provides it though)



Functions

Generic Stack-Frame

The caller also provides a reference to its own stack frame.
 ↳ This builds a linear chain of calls through the stack.
 Will be used e.g. for debugging (stack trace) and exception propagation.

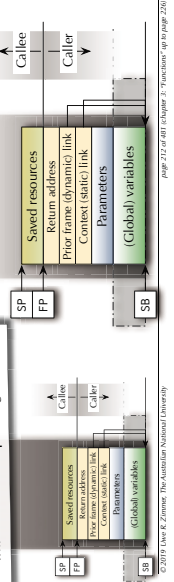


Functions

Generic Stack-Frame

A **Frame Pointer (FP)** is established at the boundary between the caller and the callee.
 ↳ Upwards from the FP: data from this function.
 ↳ Downwards from the FP: data provided by the previous function.

Saved resources are for instance registers which the callee is planning to use.

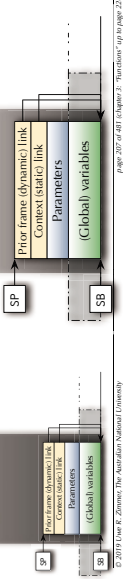


Functions

Generic Stack-Frame

The caller also provides a reference to its own stack frame.
 ↳ This builds a linear chain of calls through the stack.
 Will be used e.g. for debugging (stack trace) and exception propagation.

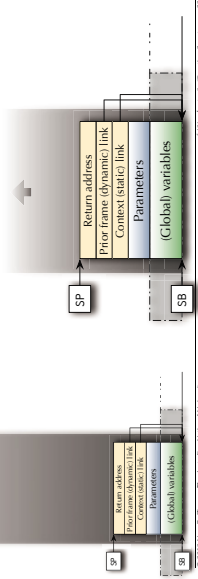
↳ The static and dynamic link might be identical in some cases.



Functions

Generic Stack-Frame

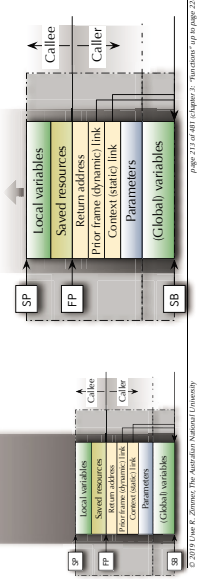
The last item to be stored before handing over control is the address of the following instruction.
 ↳ The control flow can later return to this address.



Functions

Generic Stack-Frame

Local variables are allocated (by moving the stack pointer).
 Local variables can be of any size or structure, unless the stack overflows.
 ↳ The completes a new **stack frame**.



Functions

Static vs. dynamic links

function a (x : Integer) return Integer is
 function b (y : Integer) return Integer is (x + y);
 function c (z : Integer) return Integer is (b (z));
 begin
 return c (x);
 end a;

a :: Integer -> Integer
 a x = c x
 where

b :: Integer -> Integer
 b y = x + y

c :: Integer -> Integer
 c z = b z

Dynamic link (prior frame)
 ↳ The caller of function b is function c.
 ↳ Hence exceptions raised in b are handled first in b, then in c, and then a.

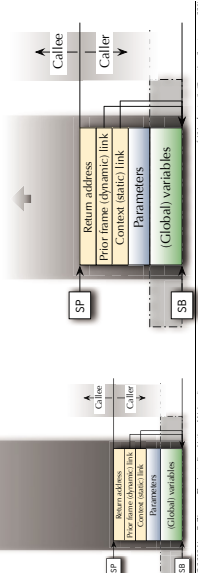
Static link (context)
 ↳ The context for function b is function a.
 ↳ Hence b can access x (but not z).

The dynamic and static link for function c are both function a.

Functions

Generic Stack-Frame

Control is handed over to the callee.
 ↳ The Instruction Pointer (IP), sometimes also called Program Counter (PC) is changed to a new address.
 Operations from here are in the control of the callee.

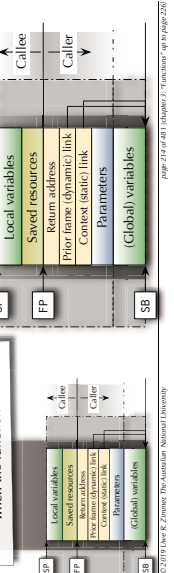


Functions

Generic Stack-Frame

Local variables are allocated (by moving the stack pointer).
 Local variables can be of any size or structure, unless the stack overflows.
 ↳ The completes a new **stack frame**.

While this function is executing, local variables can still be added.
 ↳ Handy, if e.g. the size of a local variable is not yet determined when the function starts.



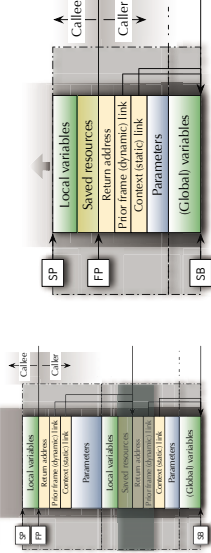
Functions

Generic Stack-Frame

The next function call will produce the next stack frame.

- Variables and parameters from the context stay visible (via the chain of static links).

Local variables can only be added to the currently executing function.



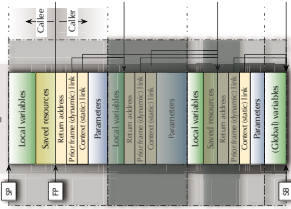
© 2019 Dave E. Zornes, The Australian National University

page 216 of 481 Chapter 7: Functions up to page 220

Functions

Generic Stack-Frame

How fast / complex is the allocation and deallocation of local variables and parameters on the stack?



© 2019 Dave E. Zornes, The Australian National University

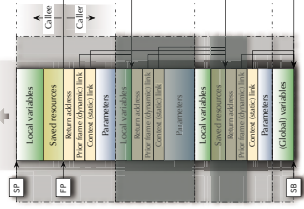
page 219 of 481 Chapter 7: Functions up to page 220

Functions

Generic Stack-Frame

The next function call will produce the next stack frame.

- Note which variables and parameters are visible.



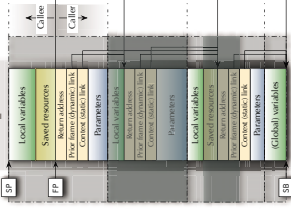
© 2019 Dave E. Zornes, The Australian National University

page 217 of 481 Chapter 7: Functions up to page 220

Functions

Generic Stack-Frame – Caller

- Pre_Call:
 - Allocate/identify space for the parameters
 - Copy the in and in-out parameters to this space
 - Potentially provide links
 - Provide a return address (Post_Call*)
 - (usually implicit with the call itself)
- Call the function
- Post_Call:
 - Copy the out and in-out parameters to local variables or registers
 - Potentially restore the frame pointer
 - Restore the stack to its previous state (if the stack has been used)



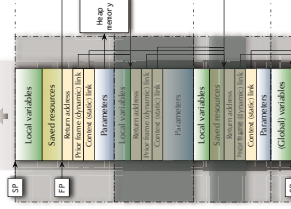
© 2019 Dave E. Zornes, The Australian National University

page 220 of 481 Chapter 7: Functions up to page 220

Functions

Generic Stack-Frame – Heap

- How to keep any memory allocation after function return?
- By using an out, by-reference parameter, the link to the newly allocated memory area is kept.



© 2019 Dave E. Zornes, The Australian National University

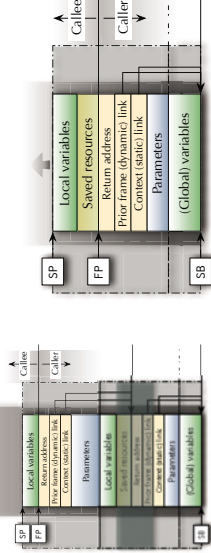
page 223 of 481 Chapter 7: Functions up to page 220

Functions

Generic Stack-Frame

The next function call will produce the next stack frame.

- Variables and parameters from the context stay visible (via the chain of static links).



© 2019 Dave E. Zornes, The Australian National University

page 215 of 481 Chapter 7: Functions up to page 220

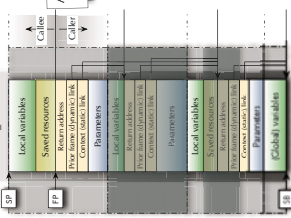
Functions

Generic Stack-Frame

The next function call will produce the next stack frame.

- Note which variables and parameters are visible.

Accessing the context like mechanism (e.g. displays, which make all context levels accessible at once).



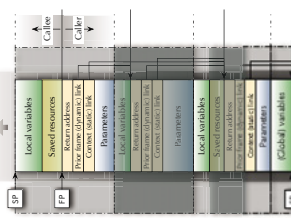
© 2019 Dave E. Zornes, The Australian National University

page 218 of 481 Chapter 7: Functions up to page 220

Functions

Generic Stack-Frame – Callee

- Prologue:
 - Save all registers which are needed inside this function to the stack
 - Establish a new frame pointer while potentially saving the previous fp
 - Allocate/identify space for local variables
 - Potentially initialize local variables
- Operations, which will use local and context variables and parameters (via the FP(s))
- Epiologue:
 - Potentially restore the prior frame pointer
 - Restore the stack to its state at entry
 - Return from function



© 2019 Dave E. Zornes, The Australian National University

page 221 of 481 Chapter 7: Functions up to page 220



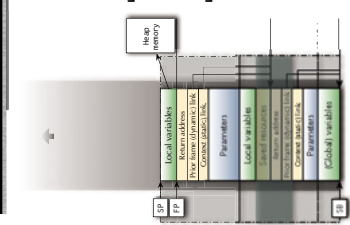
Functions

Generic Stack-Frame – Heap

⌘ How to keep any memory allocation after function return?

⌘ By using an out, by-reference parameter, the link to the newly allocated memory area is kept.

⌘ ... and a local variable in the calling function can keep this link.



© 2019 Awa K. Zimmer, The Australian National University

page 224 of 481 (Chapter 7: "Functions" up to page 226)



Functions

Generic Stack-Frame – Heap

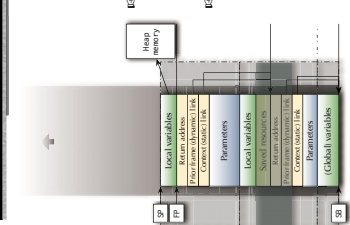
⌘ How to keep any memory allocation after function return?

⌘ By using an out, by-reference parameter, the link to the newly allocated memory area is kept.

⌘ ... and a local variable in the calling function can keep this link.

⌘ When to deallocate though?

- Garbage collection (Java)?
- Smart pointers (C++)?
- Reference ownerships (Rust)?
- Scoped pointers / storage pools (Ada)?



© 2019 Awa K. Zimmer, The Australian National University

page 225



Functions

Summary Functions

- **Framework**
 - Return address
 - Relative addressing
- **Parameter passing modes and mechanisms**
 - Copy versus reference
 - Information flow directions
 - Late evaluation
- **Stackframes**
 - Static and dynamic links
 - Parameters
 - Local variables

© 2019 Awa K. Zimmer, The Australian National University

page 226 of 481 (Chapter 7: "Functions" up to page 226)